

Weeks 3-4: Elementary data structures and their processing

Gruppenummer: 27

Gruppemedlem:	Studienummer:
August Due Iversen	202505524
Frederik Buskbjerg Reichle	202506410
Otto German Jørgensen	202500600

Exercise 1:

a)

The list is implemented as a template class that includes its own node struct. The class is based on the code from week 2, and the 8 new methods are implemented as:

void push_front(const Object x)	Pushes a node containing x just after the head
void push_back(const Object x)	Pushes a node containing x just before tail
void insert_at(Object x, int pos)	Inserts a node containing x at a given index
Object pop_front()	Removes the node located just after the head and returns its data
Object pop_back()	Removes the node located just before the tail and returns its data
Object remove_at(int pos)	Removes the node at a given index and returns its data
bool exists(Object x)	Returns true if x is in list; false if not.
void print()	Prints every nodes data in the terminal.

The code is implemented in the source code under exercise1/simple_list.h + exercise1/simple_list.cpp and tested in test_stack_class.cpp.

b)

The function is implemented as an outside function (relative to the list class), takes the reference to a list as a parameter and alters that list. Since it takes the template List class as a parameter, it utilizes the typename keyword as well. Return type is void.

Function signature:

void reverse_list(List < Object > & l)

The function creates a new temporary instance of a list, then simultaneously pushes the front popped value of the original list to the front of the temporary list, until the original is empty.

After that completes, it pushes the front popped value of the temporary list to the back of the original list.

The code is implemented in the source code under exercise1/reverse_list.h and tested in test_stack_class.cpp.

Exercise 2:

The stack is implemented as a template class using a dynamically allocated array of Objects with the following functions:

Object top()	Returns top element on the stack.
Object pop()	Pops the top element and then returns it.
void push()	Adds element to the stack. If the stack is full and new array with double the size is created, and all elements are moved into the new array.
bool empty()	Checks to see if stack is empty.
~stack()	Deleted the array.

The stack has a default size of 100 initialized in the default constructor.

The code is implemented in the source code under exercise2/stack_class.h and tested in test_stack_class.cpp.

Exercise 3:

The QueueStack class uses template and stacks. The interface has been taken from Queue_class and the functions rewritten so it fits with stacks (check queue_stack.h file)

Function	Description
bool empty()	Returns true if stack out is empty, as that is the one that holds all values.
Object front()	If the queue isn't empty it will return the value of the front.
Object get()	Unlike the front() function, this will pop the value in the front and return the same value.
void put(const Objective x)	This is the function that makes the queue function so the newest element will end at the bottom of the list. It first pops all elements into the in stack, to push the wanted x element into the out stack, which is the one that holds all elements. Afterwards we pop all the elements from the in stack into out stack again.

A main.cpp file has been created to test the QueueStack class.

Exercise 4:

Chaining:

We insert using chaining by calculating the modulus value of the key and then placing them in the corresponding buckets ranging from 0-6. We have chosen to interpret the assignment in such a way that our hash table isn't allowed to contain duplicates. Since the task doesn't specify at which load-factor the hash-table should rehash, we won't perform a rehash for this specific instance.

Example: Calculating for the first key: 5

$$h(x) = x \bmod 7$$

$$h(x) = 5 \bmod 7 = \mathbf{5}$$

Buckets:	0	1	2	3	4	5	6
Key 1:	28	15		17		5	20
Key 2:				10		19	
Key 3:						33	
Key 4:						12	

Load-factor:

$$\lambda = \frac{n}{M}$$

Where:

n = number of entries

M = number of buckets

$$n = 9, \quad M = 7$$

Load-factor for the chaining solution, given the hash function $h(x) = x \bmod 7$:

$$\lambda = \frac{9}{7} \approx 1,285714$$

Buckets:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
----------	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----

	17		19	20		5					10	28	12			15	33
--	----	--	----	----	--	---	--	--	--	--	----	----	----	--	--	----	----

After inserting all 9 values (with one of the 10 being a duplicate), we once again exceed the chosen maximum load factor of 0,5. Since we have no more values to insert, we've decided not to rehash again, but provided there was more values this would be the time to rehash.

Quadratic Probing:

We insert using quadratic probing using the same formula as before.

But now:

$$h_i(x) = (start(x) + f(i)) \bmod M$$

Where:

$$f(i) = i^2$$

Buckets:	0	1	2	3	4	5	6
	28	15				5	5

Rehashing will now be done, because the slides state that

$$\rho = \frac{\#ACTIVE - \#DELETED}{M}$$

And that it is good practice that $\rho < 0.5$ To always have a reachable empty cell

Because we have

$$\rho = \frac{4}{7} \approx 0.5714286$$

We will start rehashing. It will be done by multiplying the amount of buckets by 2 and finding the closest prime number. $7 \cdot 2 = 14$ and the closest prime number is 17.

Now it is

$$h_i(x) = (start(x) + f(i)) \bmod 17$$

Buckets	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	17		19	20	5	5			33		10	28	12			15	33

We have a load factor of

$$\lambda = \frac{11}{17} \approx 0.6470588$$

Even though it is over 0.5, it was still possible to insert all the values at empty buckets, but this is not a given in every scenario when using quadratic probing.

Exercise 5:

For this exercise we have chosen to implement the set using the queue ADT, since we can't use the internal structures or iterator, this seems to be the best option when it comes to accessing the elements of the set. Here we use pop and push to loop through them. We have implemented the following functions:

void insert()	Adds the element to the set.
void remove()	Removes the element by peaking, popping and returning on match. Pushing back when no match.
bool contains()	Checks to see if the element exists in the set.
size_t size()	Returns the size of the set.
bool isEmpty()	Checks to see if the set is empty.
void display()	Displays the elements in the set.

Since the set is unordered we can pop and push when looping through the queue, thereby changing the order of the elements.

The code is implemented in the source code exercise5/set_class.h and tested in exercise5/main.cpp.

Exercise 6:

For this exercise we have implemented a dictionary using a vector of pairs. With the pairs representing keys and values.

We have implemented the following functions:

void insert()	Pushes a key/value pair to the back of the vector if the key isn't already contained. If the key is contained, it overwrites the value.
void remove()	Loops through the vector until it find the pair with the matching key, then erases it.
bool contains()	Loops through the vector, if it finds a pair with a matching key, it returns true. If the loop finishes it returns false.
Valuetype get()	Loops through the vector until it finds a pair with a matching key, then returns the value from that pair. If the loop finishes, it throws and error.
size_t size()	Returns the size of the vector.
bool isEmpty()	Checks if size of vector is 0, if it is it returns true - if not, then false is returned
void display()	Prints all the pairs to the terminal.

The code is implemented in the source code exercise6/dictionary.h and tested in exercise6/main.cpp.

Exercise 7:

The first table:

Index	Value
0	22
1	
2	
3	
4	
5	5
6	16
7	
8	
9	27
10	

Verifying table:

Looking at the hash table we can conclude that the insertions are correct using quadratic probing. The first two values 22 and 15 are inserted seamlessly into empty indexes using:

$$h(x) = x \bmod 11$$

The next value 16 has modulus:

$$h(x) = 16 \bmod 11 = 5$$

Since that index is filled the collision resolution is given by:

$$h_i(x) = (start(x) + f(i)) \bmod M$$

Where:

$$f(i) = i^2$$

Therefore:

$$h_1(x) = (16 + 1^2) \bmod 11 = 6$$

Therefore, the value 16 now lands at index 6.

For the next value 27 the modulus is:

$$h(x) = 27 \bmod 11 = 5$$

Once again index 5 is still filled and now:

$$h_1(x) = (16 + 1^2) \bmod 11 = 6$$

Since it's also taken we increase i:

$$h_1(x) = (16 + 2^2) \bmod 11 = 9$$

The value 27 correctly lands at index 9.

To summarize, the table is correct as it is given.

Adding elements:

We add the following elements, using August Due Iversen.

- 1
- 12
- August Age = 23
- August Studienummer = 202505524

1:

1 is inserted into index 1.

12:

Then 12 has index 1 which is taken.

Then the following:

$$h_1(x) = (12 + 1^2) \bmod 11 = 2$$

12 is inserted to index 2.

23:

23 has index 1 which is taken.

Then the following:

$$h_1(x) = (23 + 1^2) \bmod 11 = 2$$

2 is also taken.

Then the following:

$$h_1(x) = (23 + 2^2) \bmod 11 = 5$$

5 is also taken.

Then the following:

$$h_1(x) = (23 + 3^2) \bmod 11 = 10$$

23 is inserted to index 10.

202505524:

202505524 has index 1. As with before 2, 5 and 10 is taken.

Then the following:

$$h_1(x) = (202505524 + 4^2) \bmod 11 = 6$$

6 is taken.

Then the following:

$$h_1(x) = (202505524 + 5^2) \bmod 11 = 4$$

202505524 is inserted to index 4.

Now we have the following final table with all new elements inserted:

Index	Value
0	22
1	1
2	12
3	
4	202505524
5	5
6	16
7	
8	
9	27
10	23